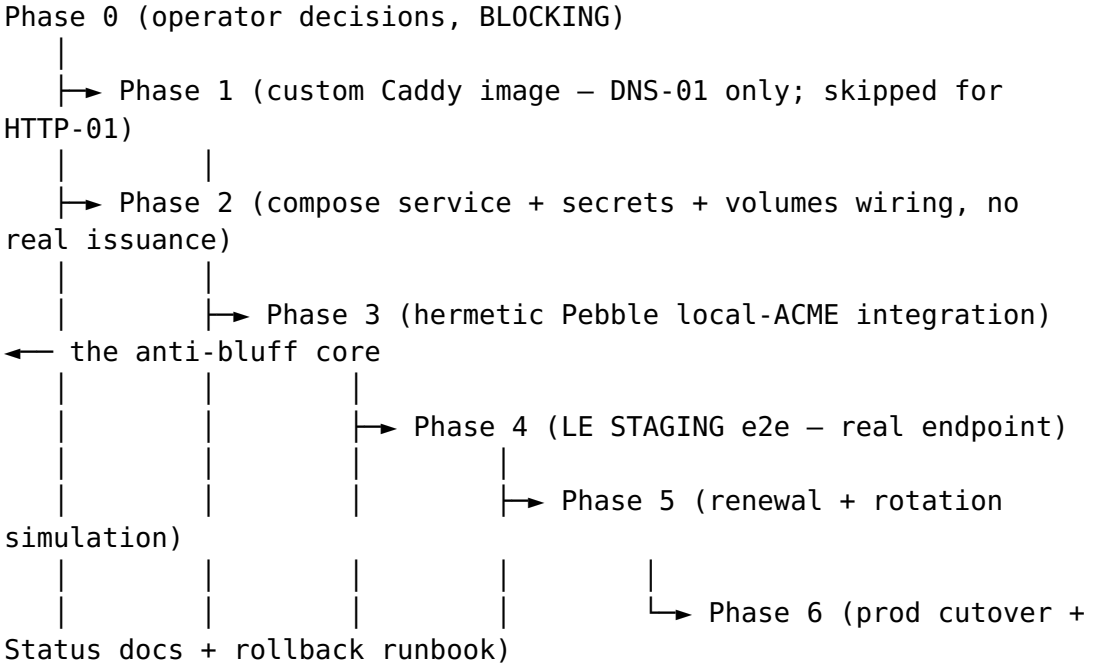
Let's Encrypt HTTPS — Phased Implementation Plan

Revision: 1 **Last modified:** 2026-07-01T11:00:00Z **Status:** Draft — implementation plan for the design in LETSENCRYPT_HTTPS.md. Phase 0 is BLOCKING on the OPERATOR INPUT REQUIRED decisions (design §9). **Authority:** Inherits `constitution/Constitution.md` per §11.4.35. Phases are PWU-sized (§11.4.58): each self-contained, each with files + tests + a §11.4.135 guard + a §1.1 mutation + captured evidence (§11.4.5/§11.4.69).
Companion design: LETSENCRYPT_HTTPS.md

> Docs-only planning artifact. No code, no boot, no commits produced here. Every phase below is TDD-first (§11.4.43/§11.4.115): write the RED test that reproduces "feature absent/broken", then implement to GREEN, then register the guard + mutation. Non-unit tests boot infra via the containers submodule (§11.4.76) and SKIP-with-reason when topology is absent (§11.4.3), never fake-PASS.

Dependency graph (order is load-bearing)



Phases 1–5 use **staging / Pebble only** — zero production rate-limit exposure (design §3.3). Phase 6 is the single operator-gated production issuance.

Phase 0 — Operator decisions + tracker scaffolding (BLOCKING)

Goal: resolve the three OPERATOR INPUT REQUIRED decisions (design §9: hostname, challenge type, staging/prod + `:443` reachability) and open the workable items.

- **Files:** `docs/Issues.md` (+ summary) entries with ATM-NNN ids per §11.4.54; a `docs/design/letsencrypt/Status.md` + `Status_Summary.md` scaffold (§11.4.45/§11.4.56) marked PENDING.
- **Tests/guard:** none (planning phase). Guard is documentary: the tracker entries exist with Type/Status/ATM-id (§11.4.148).
- **Evidence:** the `AskUserQuestion` decision record captured in the tracker.
- **Exit:** Decisions A/B/C answered. Decision B (DNS-01 vs HTTP-01) selects whether Phase 1 runs.
- **Depends on:** nothing. **Blocks:** everything.

Phase 1 — Custom Caddy image with the DNS-01 provider module (DNS-01 branch only)

Goal: produce a rootless-runnable Caddy image that contains the operator-chosen `caddy-dns/<provider>` module (the stock image lacks DNS modules — design §4.3). **Skipped entirely if Decision B = HTTP-01** (stock image suffices).

- **Files:**
  - `config/caddy/Containerfile` — `xcaddy build` with `--with github.com/caddy-dns/<provider>` (mirrors `config/squid/Containerfile.dynamic`).
  - build recipe wired through the containers submodule `pkg/crossbuild` (no ad-hoc podman build — §11.4.76).
  - `.env.example` — `CADDY_DNS_PROVIDER`, `SQUID`-style image tag var `CADDY_IMAGE`.
- **Tests (§11.4.169):**
  - unit: Containerfile lints / `sh -n` recipe (§11.4.67); RED = image built WITHOUT the module.
  - integration: build the image via `pkg/crossbuild`; assert `caddy list-modules | grep dns.providers.<provider>` present (§11.4.38 open-the-artifact evidence).

- **Guard §11.4.135:** `caddy-image-has-dns-module`.
- **§1.1 mutation:** drop the `--with` line → guard FAILs.
- **Evidence:** `caddy list-modules` output under `qa-results/<run-id>/letsencrypt/phase1/`.
- **Depends on:** Phase 0 (Decision B). **Blocks:** Phase 2 (DNS-01 branch).

Phase 2 — Compose service, Podman secret, and cert volume wiring (no real issuance)

Goal: define `proxy-caddy` and its secret/volume plumbing so the front boots and terminates TLS with a *self-signed/internal* cert — proving the wiring before any ACME.

- **Files:**
  - `docker-compose.https.yml` — new overlay (§11.4.122 pattern) with `proxy-caddy`, its dedicated Podman network, `:443`/`:80` publish per Decision C-port, `secrets:` for `CADDY_DNS_TOKEN_SECRET`, volumes `caddy-data` + `caddy-config`.
  - `config/caddy/Caddyfile` — upgrade `:80 {}` → `{$CADDY_DASHBOARD_DOMAIN}` site block with `tls` issuer stub set to Caddy's **internal** CA first (no ACME yet), keep `/health` / `status` `file_server`, add `:80`→`:443` redirect.
  - `.env.example` — all new CADDY_* vars (design §4.3).
  - `.gitignore` — `caddy-data/`, `caddy-config/`, `*.pem`/`*.key` under caddy paths (§11.4.30) + a `.gitignore-meta` regen note (§11.4.77: certs re-issued via ACME).
  - containers-submodule `pkg/boot`/`pkg/compose` entry + `pkg/health` readiness (TLS handshake + `/health` 200).
- **Tests (§11.4.169):**
  - unit: Caddyfile env-substitution render; secret-NAME-not-value assertion.
  - integration: boot `proxy-caddy` (internal CA); assert an HTTPS `/health` 200 with Caddy's internal cert; assert the DNS token is present ONLY as `/run/secrets/…` (never in the rendered config).
  - security: leak-grep — the token value is absent from tree + history (§11.4.10.A).
- **Guard §11.4.135:** `caddy-front-serves-https-health` + `dns-token-is-secret-only`.
- **§1.1 mutations:** (a) inline the token value into compose → leak guard FAILs; (b) drop the `tls`/redirect → https-health guard FAILs.
- **Evidence:** `openssl s_client` handshake + `/health` capture; leak-grep output.
- **Depends on:** Phase 1 (DNS-01) / Phase 0 (HTTP-01). **Blocks:** Phase 3.

Phase 3 — Hermetic local-ACME integration with Pebble (anti-bluff core)

Goal: Caddy issues a **real ACME cert against a local Pebble** server with the chosen challenge solved locally — hermetic, no network, no rate limits (design §3.4).

- **Files:**
  - `tests/letsencrypt/pebble_integration_test.*` — boots Pebble via the containers submodule; points Caddy `CADDY_ACME_CA` at Pebble's directory; solves DNS-01 against a local test resolver (or HTTP-01 against Pebble's challenge port).
  - a cert-chain/expiry **analyzer** helper with **golden-good + golden-bad** fixtures (§11.4.107(10)).
- **Tests (§11.4.169):**
  - integration (hermetic): assert Caddy serves a leaf whose chain roots in Pebble's test CA; assert SAN == chosen hostname.
  - unit: expiry-math + chain-verify analyzer table tests; golden-bad (expired / wrong-CA) MUST FAIL.
  - negative: point Caddy at a bogus ACME dir → issuance fails loudly (no plaintext fallback).
- **Guard §11.4.135:** `acme-issues-cert-hermetic` + analyzer self-validation `cert-analyzer-golden-good-bad`.
- **§1.1 mutations:** (a) bogus ACME dir → hermetic guard FAILs; (b) make the analyzer accept the golden-bad fixture → self-validation FAILs.
- **Evidence:** served leaf + chain PEM (public only), Pebble logs, analyzer verdicts under `qa-results/<run-id>/letsencrypt/phase3/`.
- **Depends on:** Phase 2. **Blocks:** Phase 4.

Phase 4 — Let's Encrypt STAGING end-to-end (real endpoint, real DNS-01)

Goal: issue a **staging** cert for the real operator hostname against `acme-staging-v02` — real ACME protocol, real DNS TXT write/clear, no production rate-limit exposure.

- **Files:**
  - `tests/letsencrypt/staging_e2e_test.*` — sets `CADDY_ACME_CA` = staging; runs the real DNS-01 flow using the Podman-secret token; SKIP-with-reason if the token/topology is absent (§11.4.3, never fake-PASS).

- **Tests (§11.4.169):**
  - integration (staging): assert a staging-issued cert (valid shape, untrusted root) for the hostname; assert the `_acme-challenge` TXT was written AND cleared (§11.4.14 cleanup).
  - e2e: real HTTPS GET to `/health` → 200 over the staging cert; capture SANs + issuer.
- **Guard §11.4.135:** `acme-staging-issues-for-hostname`.
- **§1.1 mutation:** revoke/omit the DNS token → staging issuance fails (guard asserts loud failure, not silent PASS).
- **Evidence:** `openssl s_client` chain (issuer = LE staging), DNS TXT before/after, `/health` 200.
- **Depends on:** Phase 3 + operator token (Decision B/C). **Blocks:** Phase 5.

Phase 5 — Renewal + rotation simulation (zero-downtime)

Goal: prove the built-in renewal loop **fires before expiry** and the front **serves the NEW chain with zero downtime** (design §3.3, §6).

- **Files:**
  - `tests/letsencrypt/renewal_sim_test.*` — seeds a **fake near-expiry cert** into `caddy-data`, drives renewal against Pebble/staging, holds a live TLS connection across the swap.
- **Tests (§11.4.169):**
  - renewal-simulation: seed near-expiry `NotAfter`; observe renewal; assert new `NotAfter` > old.
  - rotation: assert served leaf **serial changed** after renewal AND the held connection was NOT dropped (zero-downtime).
  - chaos (§11.4.85): kill/restart Pebble mid-renewal → Caddy retries with backoff, keeps serving the still-valid current cert (no outage while valid).
  - stress: N renewals back-to-back → identical outcome (§11.4.50 determinism).
- **Guards §11.4.135:** `renewal-fires-before-expiry` + `rotation-serves-new-chain` + `failclosed-no-plaintext-fallback`.
- **§1.1 mutations:** (a) freeze the renewal loop → renewal guard FAILs; (b) pin the old cert (skip swap) → rotation guard FAILs; (c) add a plaintext fallback on issuance failure → failclosed guard FAILs.
- **Evidence:** before/after `NotAfter` + serials, held-connection trace, chaos retry log.
- **Depends on:** Phase 3 (Pebble) / Phase 4 (staging). **Blocks:** Phase 6.

Phase 6 — Production cutover + Status docs + rollback runbook

Goal: the single operator-gated production issuance, plus the durable docs + standing guards.

- **Files:**
  - flip `CADDY_ACME_CA` → production (operator-approved change; one-shot, rate-limit aware — design §3.3).
  - `docs/design/letsencrypt/Status.md` + `Status_Summary.md` finalized PASS/FAIL with evidence paths (§11.4.45/§11.4.56); README doc-link row (§11.4.57); CONTINUATION §3 update (§12.10).
  - rollback runbook section (design §7.2) verified.
  - register all Phase 1–5 guards into the standing regression-guard suite (§11.4.135); Challenges/HelixQA bank entry (§11.4.169).
- **Tests (§11.4.169):**
  - e2e (prod): real browser-trusted HTTPS to `/health` → 200; capture the **trusted** chain (issuer = LE production).
  - full §11.4.40 retest of the whole suite on a clean baseline before any release tag.
- **Guard §11.4.135:** `acme-prod-cert-trusted-for-hostname` (standing).
- **§1.1 mutation:** point prod issuer at staging → trusted-chain guard FAILs (proves it checks the real root).
- **Evidence:** trusted-chain capture, full-suite green log, Status docs synced (§11.4.60/§11.4.65 exports).
- **Depends on:** Phase 5 + operator prod approval (Decision C). **Blocks:** release tag.

Alternative branch — if the operator rejects Caddy and picks lego/certbot

Documented for completeness (design §3.1 recommends Caddy). If chosen, the renewal mechanism changes and MUST still honor §11.4.156 (no cron-in-CI, no git hook):

- Renewal runs as a **`systemd --user` timer** on the host (operator-installed), OR a **long-running renewal container/loop** — NOT cron-in-CI, NOT a git hook.
- Add a **reload hook**: on successful renewal, reload the TLS terminator (SIGHUP or a containers-submodule `pkg/health`-gated restart) — because lego/certbot write cert FILES that a separate terminator must pick up (no in-process hot-swap like Caddy).

- All Phase 3–5 tests still apply (Pebble hermetic, staging, renewal/rotation), retargeted at the lego/certbot + terminator pair.
- This branch adds moving parts (timer + reload hook + separate terminator) versus Caddy's single self-renewing process — the reason Caddy is recommended.

Cross-cutting requirements (apply to every phase)

- **Anti-bluff (§11.4/§11.4.69):** every PASS via `ab_pass_with_evidence <desc> <path>`; topology-absent ⇒ `ab_skip_with_reason` (§11.4.3), never fail-open.
- **TDD (§11.4.43/§11.4.115):** RED-on-broken (bogus ACME / missing module / near-expiry) → GREEN, with a `RED_MODE` polarity switch where practical.
- **containers submodule (§11.4.76):** Pebble/step-ca/Caddy all booted via `pkg/boot`/`pkg/compose`/`pkg/health`; no ad-hoc podman.
- **secrets (§11.4.10/§11.4.30):** token = Podman secret NAME only; certs/account-key gitignored; pre-store leak audit (§11.4.10.A).
- **docs sync (§11.4.60/§11.4.65/§11.4.106):** every doc change regenerates `.html`/`.pdf`; Status docs kept in sync.
- **review (§11.4.142/§11.4.125/§11.4.134):** every change through independent review, iterate to clean GO.
- **CONST-033 / no host power / no host network by the implementer:** the `:443` sysctl/NAT and the Podman secret creation are **operator-performed host steps**, documented in the runbook, never executed autonomously.